Final Project Report

Tayarny Airlines — Flight Reservation System

Team Members:

Abdelrahman Amr (9010)
Omar Waleed (8873)
Youssef Mohamed (8821)
Mohamed Hatem (8995)
Mahmoud Waleed (8958)
Abdelrahman Huessin (8976)

May 13, 2026

Contents

1 Project Overview **3**

2 Problem Statement **3**

3 System Scope **3**
3.1 In Scope 3
3.2 Out of Scope 3

4 Requirements **3**
4.1 Functional Requirements 3
4.2 Non-Functional Requirements 3

5 Software Development Lifecycle (SDLC) **4**
5.1 Design Phase 4
5.1.1 Use Case Diagram 4
5.1.2 Sequence Diagram 4
5.1.3 Activity Diagrams 4
5.1.4 Class Diagram 4
5.1.5 State Machine Diagrams 6
5.2 Implementation and Testing 8

6 Project Management & Collaboration **8**

7 Maintenance and Future Work **8**

1 Project Overview

Tayarny Airlines is a full-stack flight reservation system that supports a complete booking lifecycle for two roles: **Customer** and **Administrator**. The system is implemented with a Next.js frontend, a Spring Boot REST API backend secured by JWT, and a PostgreSQL database.

2 Problem Statement

Traditional flight booking and reservation management can be slow and error-prone. The problem addressed by this project is building a secure, end-to-end reservation platform that provides a simple customer booking workflow, maintains correct seat availability under concurrent usage, and gives administrators controlled access to operational data.

3 System Scope

3.1 In Scope

- User authentication and authorization (Customer/Admin).
- Flight search, fare options, seat map selection, and traveler data entry.
- Reservation history, details, and cancellation rules.
- Admin management of flights, airports, airplanes, and users.

3.2 Out of Scope

- Real payment gateway integration (simulated only).
- Airline check-in, boarding pass generation, and baggage handling.
- Loyalty programs and dynamic pricing.

4 Requirements

4.1 Functional Requirements

The system includes comprehensive functional requirements ranging from user registration (FR-1) and JWT-based login (FR-2) to complex seat map visualization (FR-10), double-booking prevention (FR-13), and full Admin CRUD capabilities (FR-19 to FR-24).

4.2 Non-Functional Requirements

Key performance metrics include sub-2-second response times (NFR-1). Security is prioritized via BCrypt password hashing (NFR-3) and stateless JWT sessions (NFR-4). Reliability is ensured through database locking for seat assignments (NFR-11).

5 Software Development Lifecycle (SDLC)

5.1 Design Phase

5.1.1 Use Case Diagram

The Use Case diagram provides a high-level overview of the system's functions, showing how both Customers and Administrators interact with the booking and management modules.

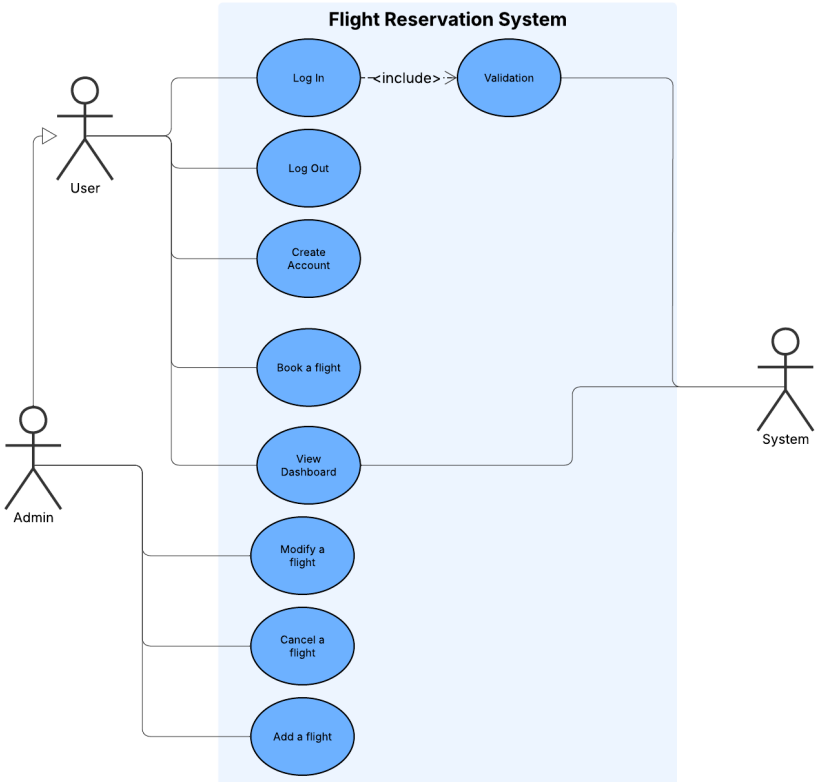


Figure 1: Use Case Diagram illustrating Customer and Admin interactions.

5.1.2 Sequence Diagram

This diagram details the chronological sequence of interactions between the frontend, backend services, and the database during the flight booking process.

5.1.3 Activity Diagrams

The following diagrams describe the step-by-step logical workflows for key system processes, ensuring clear transitions between user actions and system responses.

5.1.4 Class Diagram

The Class Diagram maps the static structure of the system, showing the attributes and methods of entities like Users, Flights, and Reservations, along with their relationships.

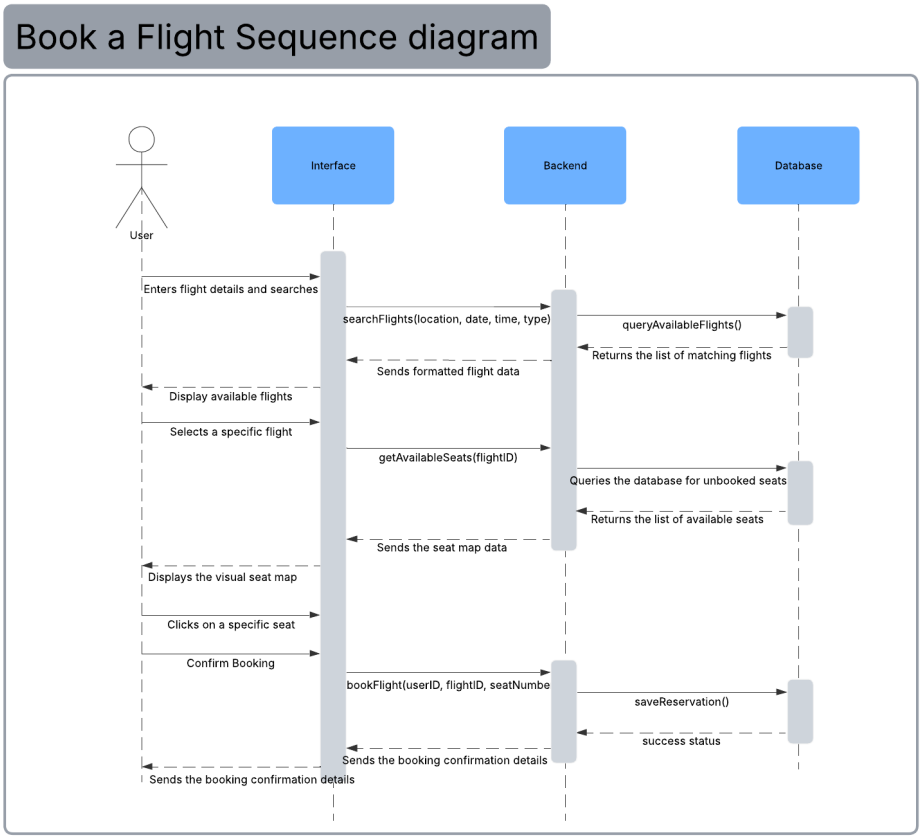
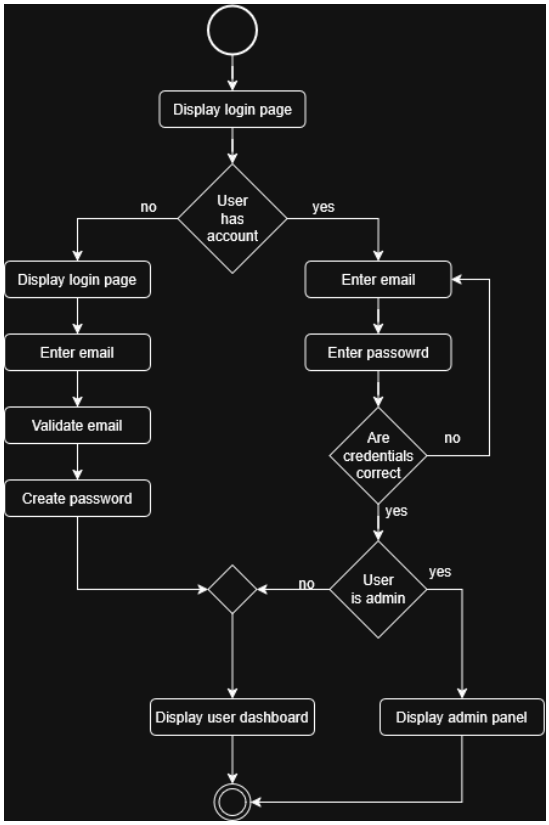


Figure 2: Sequence Diagram showing the flow of booking messages between layers.



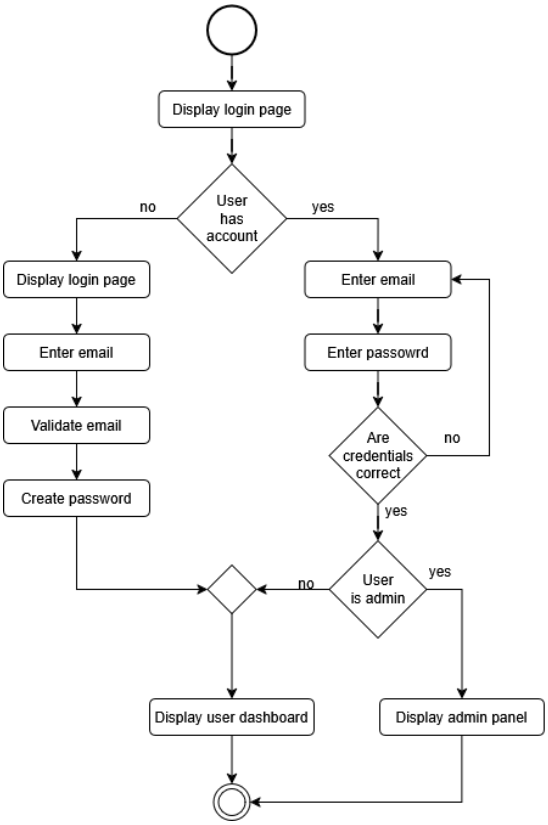
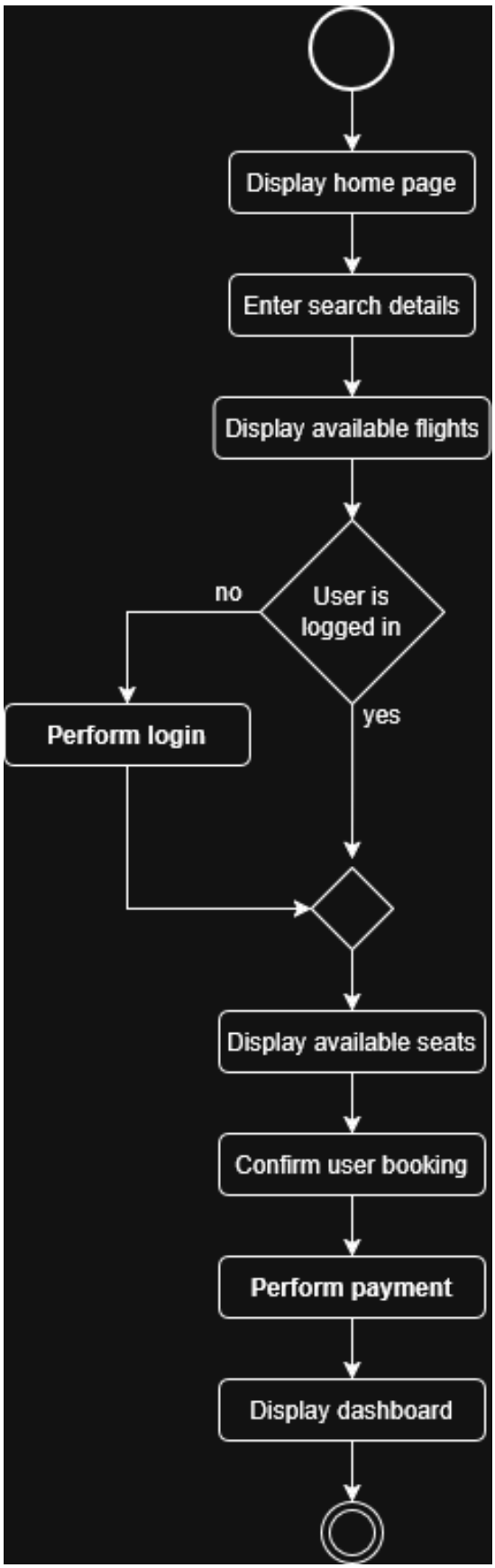


Figure 3: User Login logic flow (Phase 1 and 2), detailing authentication and token generation.

5.1.5 State Machine Diagrams

These diagrams track the lifecycle of system entities, specifically focusing on how flight and seat statuses change based on external triggers like cancellations or completions.



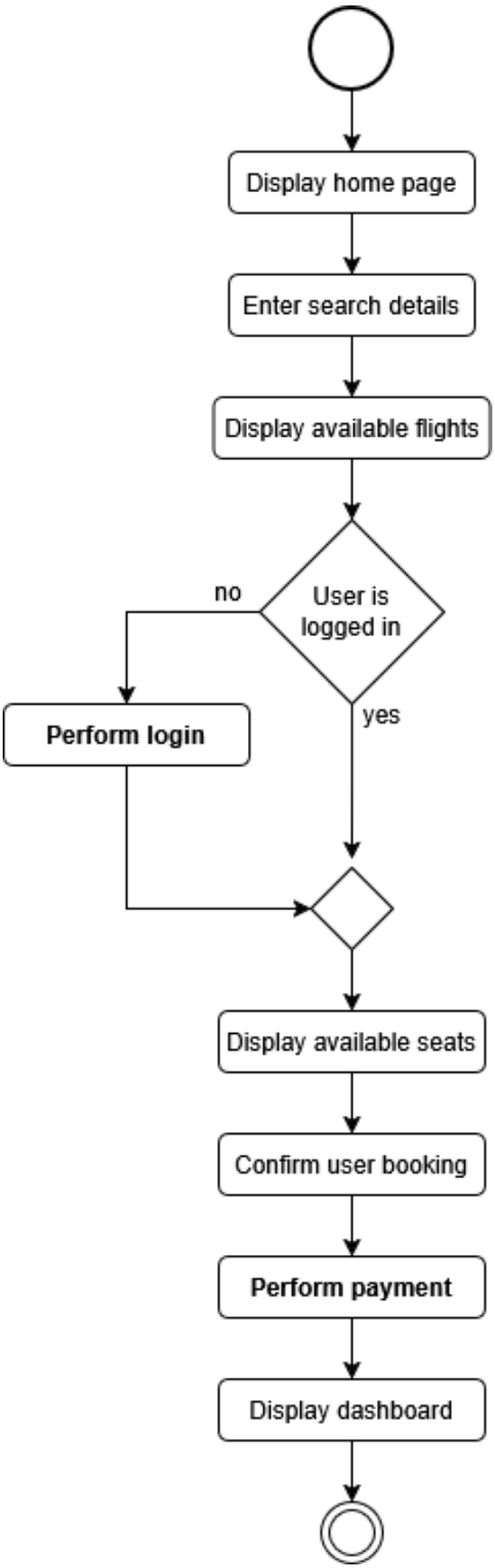
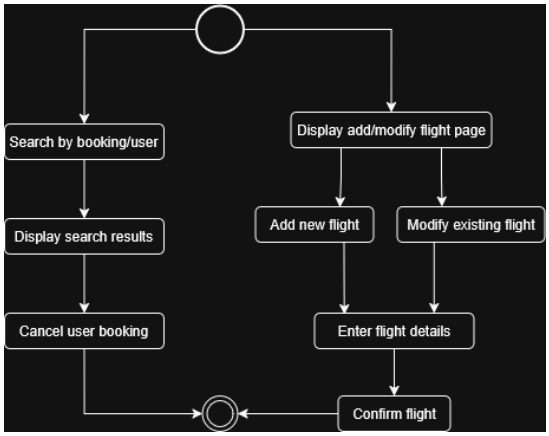


Figure 4: Flight booking activity workflow from search to initial selection.



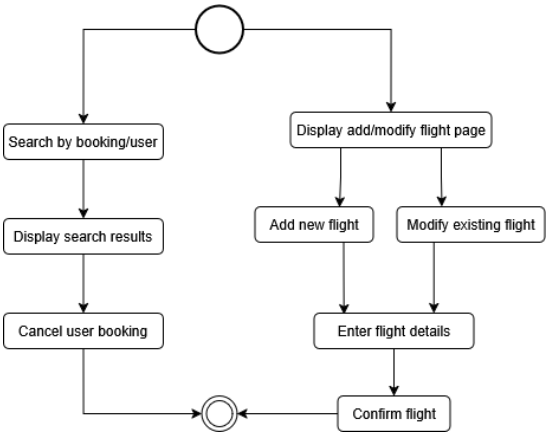


Figure 5: Administrative panel management process for flight and user oversight.

5.2 Implementation and Testing

The system was built using Java 25 and Spring Boot. Automated testing was conducted via JUnit 5, while manual verification was documented through various script-based tests. The file directory for these assets is visible in **image_3ac63a.png**.

6 Project Management & Collaboration

The team utilized Trello for task tracking and Notion for centralized documentation, ensuring a smooth SDLC progression.

7 Maintenance and Future Work

The modular architecture ensures high maintainability. Future updates will focus on real payment gateway integration and automated notification services (Email/SMS).

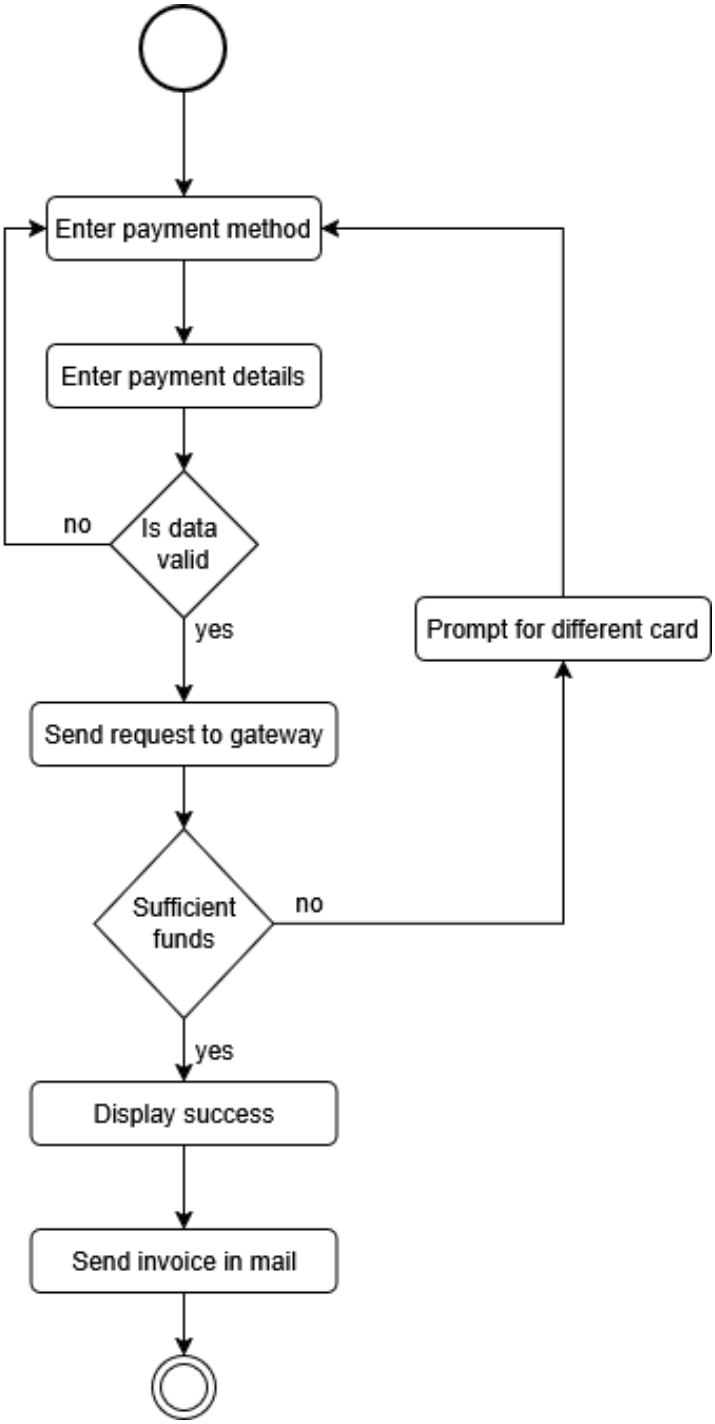


Figure 6: Simulated payment validation workflow ensuring secure checkout logic.

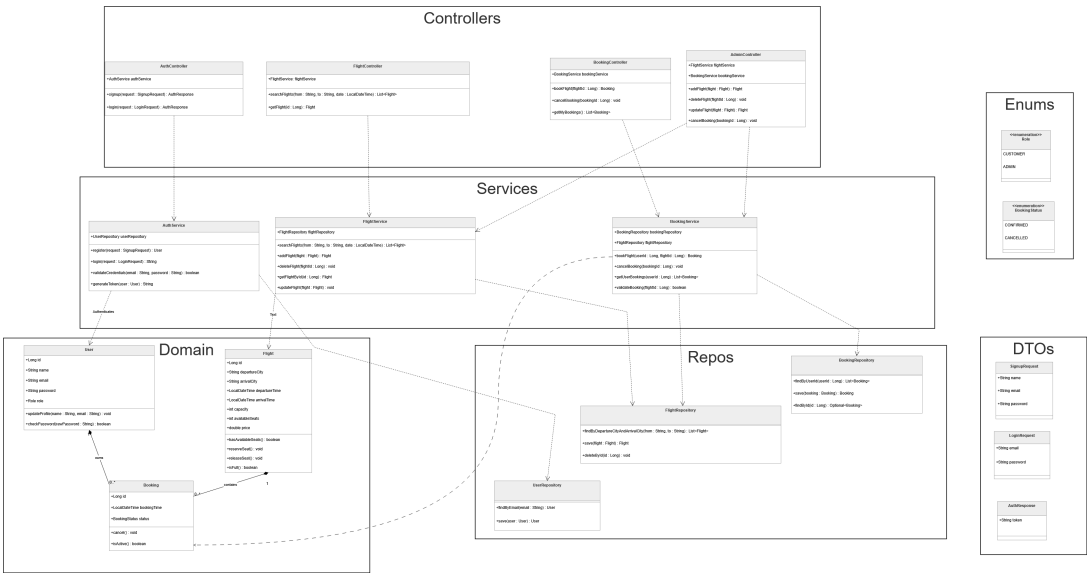


Figure 7: System Class Diagram showing entity relationships and data structures.

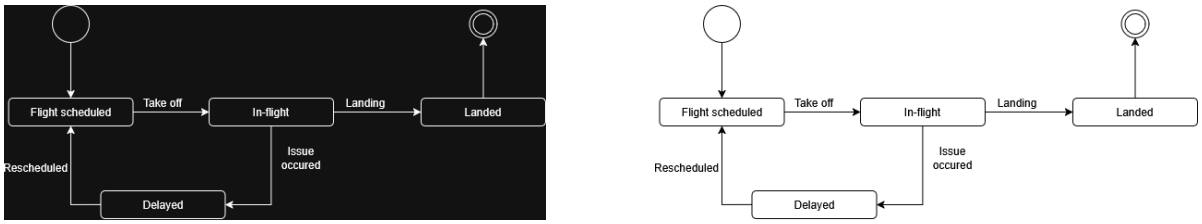


Figure 8: Flight Status lifecycle transitions from scheduled to completed or cancelled.

Figure 9: Seat Reservation status transitions, managing availability and locks.

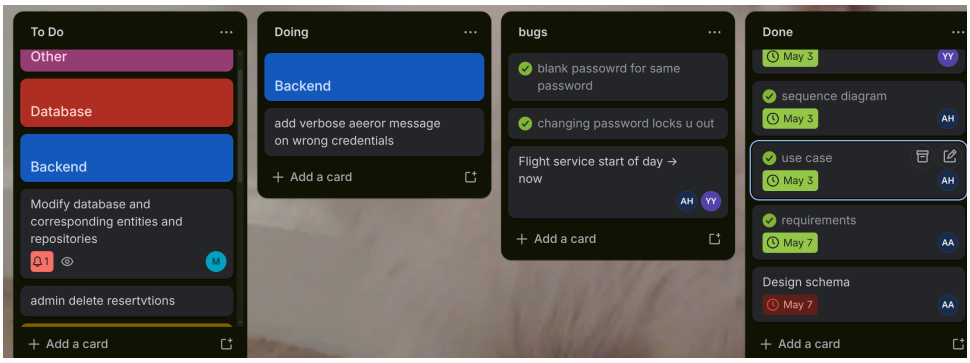


Figure 10: Trello board used for sprint planning, task assignment, and progress tracking.